

EMUQU — A Measurement-Grounded Assistant with a Multilingual Regulated-Claim Perimeter

Typed fact catalogue, runtime number verification, and one 17-language vocabulary enforced at input, output and build time

Technical White Paper • v1.0

October 2026

Chris Sharp

Describes Emuqu at source revision e028039 (2026-10-07); scoring version v3.1.oct2026.

Executive Summary

Emuqu is an iOS heart-rate-variability and training app with a conversational coach ("Flo") that can run on six language-model providers: five hosted (Anthropic, OpenAI, Google Gemini, xAI Grok, DeepSeek) and one on-device (Apple Intelligence). Two problems dominate an assistant of this kind. First, it must quote the user's own measurements correctly. A model that says "your TSB is -29" while the dashboard shows -15.5 is wrong in a way the user can check. Second, it must stay inside a wellness perimeter in every language the app ships. That means no naming of a condition for the user, no rhythm assessment, no treatment or dosing, and no categorical certainty about physiological state. The user must also not be left without help when they report an emergency, an injury or thoughts of self-harm.

This paper describes the methods Emuqu uses for both problems, in enough detail to reimplement them:

1. **A typed fact catalogue.** Every fact the model can read is one declarative entry. The entry binds a key grammar, a description, a value type, an availability check and the resolver closure, so a resolver cannot exist unless it is also published. Results come back in a single JSON envelope with a closed, nine-value absence taxonomy. The same envelope is used for all six providers.
2. **A compact tool layer.** 21 polymorphic read tools and 18 allow-listed action tools are drawn from the catalogue. They are sorted and de-duplicated so the bytes stay stable, and a capability index is generated from the same specification list.
3. **Runtime number verification.** Numeric claims about the value *now* are checked against app state with per-metric tolerances. Spelled-out numbers are normalised first. Only the claimed span is corrected, and the correction is fed into the next turn's prompt.
4. **Structural conflict resolution.** When the user contradicts cached numbers or states their own, the stale dashboard block is removed from the prompt rather than argued with.

5. **One multilingual regulated-claim lexicon.** The lexicon has 29 concepts in 17 languages. It drives three enforcement points: a pre-send input guard that answers locally with fixed, localized replies and never sends the turn to any provider; a sentence-level, negation-neutral output guard applied before text is displayed or spoken; and a build-time linter over Swift source and every locale of the string catalogues. A CI gate proves, using a witness string per pattern, that the runtime lexicon covers the build-time list.

What is new is the combination and the specific mechanisms listed in Section 4. Function calling, retrieval, prompt caching, BM25 and regex filtering are prior art and are not claimed. The perimeter is lexical rather than semantic, and Section 5 lists its known gaps.

1. The Problem

1.1 Wrong numbers that look right

Large language models produce fluent text that can contradict their inputs. This is the failure the hallucination literature calls intrinsic or input-conflicting hallucination [1]. In a health-data assistant it shows up as concrete, checkable errors. In logs this app recorded, a model quoted training stress balance (TSB) values from -9 to -29 across turns while the dashboard showed one stable value. In voice mode, a model stated a heart rate of 72 bpm while the live value was 91. These errors come from several sources:

- **The model guesses instead of retrieving.** It answers from priors, or from a value in an earlier turn.
- **Several sources disagree.** Different tools return values computed at different times, for example a readiness snapshot frozen at workout start versus the current load. The model picks the wrong one.
- **Stale context.** A cached block in the prompt contradicts what the user has just said. Benchmarks of stale-premise rejection suggest prose instructions ("trust the user") are not reliable. This is why the method in 3.6 removes the stale block instead of instructing the model.
- **Spoken numbers.** In voice mode the model writes "negative thirty-one point seven", which no digit-matching check sees.

Retrieval-augmented generation [2] and tool use [3][4] reduce guessing, but they do not check what the model finally says. Post-hoc verification methods such as RARR [5] and FActScore [6] check claims against evidence using further model calls. That adds latency and cost and is not deterministic. For a streaming voice coach, a check has to run in milliseconds on each sentence before it is spoken.

1.2 A regulated perimeter in seventeen languages

The app is a general-wellness product and not a medical device. US FDA general-wellness guidance [7] and App Store Review Guideline 1.4.1 [8] both turn on what the product claims. A model that tells a user "you may have atrial fibrillation" makes a claim the app cannot support. A model that tells a user in crisis to "see a doctor" without giving a crisis line fails them.

Known approaches have gaps for this case:

- **System-prompt rules** are ignored or truncated by some models, and can be overridden by injection [9].

- **Classifier guardrails** such as Llama Guard [10] or programmable rail frameworks such as NeMo Guardrails [11] need a model call or a hosted service. Their taxonomies are generic safety categories, not a regulated-claim vocabulary, and they do not run on the iPhone between token arrival and speech.
- **English-only keyword lists** fail silently. The app ships 16 non-English localizations, and the assistant answers in the user's language. Before the shared lexicon existed, the input guard knew 2 patterns, the output guard 12 and the build-time list 21, all in English, with nothing reconciling them.
- **Negation handling** (NegEx [12] and successors) is designed for clinical notes in one language. It is not robust to sentence-final negation in Japanese and Korean or to split negation in French. A missed negation lets a claim through.
- **Static copy and model output are governed separately.** A term forbidden in static copy can still be said by the model, and the reverse.

2. Architecture Overview

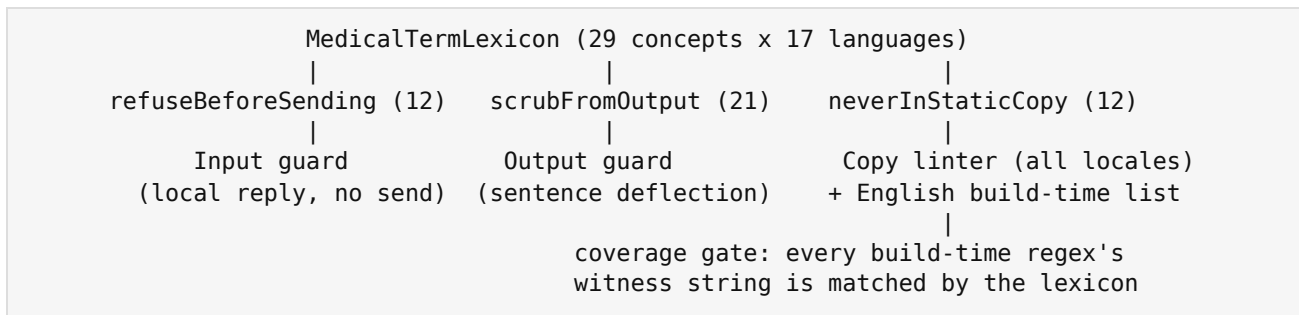
Table 1 shows one user turn on the text path. The voice path is the same up to the provider call. After that, each speakable chunk goes through steps 10 and 11 before speech.

Table 1. Turn pipeline

Phase	Operation	Purpose
1	Trim input; reject empty	—
2	Input guard: classify the current message against the lexicon's refuse groups in safety order	Answer emergencies, injuries, self-harm and rhythm questions locally; nothing is sent
3	Consent check for the active hosted provider	No health data goes to a provider the user has not consented to
4	History assembly: drop <code>localOnly</code> turns, truncate to the provider budget, summarise dropped turns	Guarded turns never leave the device on a later send
5	Build or reuse the fact registry; reset per-turn rate-limit counters	One address space for all facts
6	Compose tool list: 21 read tools + allow-listed actions, de-duplicated and sorted	Byte-stable schema
7	Compose system prompt: stable prefix (rules, tool overlay, capability index) + variable sections (time, user-asserted values, dashboard cache unless suppressed, live-workout marker, pending corrections, summary)	Grounding, conflict resolution, correction loop
8	Stream the provider round; tool calls go to the router → registry → envelope, at most 8 per turn	Retrieval with uniform absence semantics
9	Output guard, streaming: publish only complete sentences, each scrubbed	Prohibited sentences are never displayed
10	Number verification at each round end (chat) / per spoken chunk (voice)	Replace contradicted claimed spans; record corrections
11	Output guard, speech: scrub each utterance before TTS	Nothing prohibited is spoken

Phase	Operation	Purpose
12	Output guard, final pass on the whole message	Covers text assembled outside the stream
—	Build time: copy linter over Swift and string catalogues; lexicon coverage gate	Static copy follows the same vocabulary

Figure 1 shows the shared-vocabulary structure.



3. Methods in Detail

3.1 The typed fact catalogue

Entry kinds. A fact is a value of a four-case sum type `FactEntry`:

Kind	Key	Resolver signature	Notes
fixed	literal dotted key, e.g. <code>user.profile.max_hr</code>	<code>() -> FactValue</code> , body is <code>.sync</code> or <code>.awaitable</code>	Carries <code>valueType</code> ("Int", "Double", "Duration", "Bool", "Record"...)
parameterized	pattern with one <code>\$param</code> , e.g. <code>session.by_date(\$date)</code> plus an example argument	<code>(param, tail) -> FactValue</code>	Matches on leading tokens; trailing tokens pass as <code>tail</code>
composite	literal or parameterised	<code>(param?, registry) -> FactValue</code>	Declares <code>dependencies</code> (atomic keys only)
action	dotted verb key, e.g. <code>routes.library.rename</code>	<code>([String:String]) -> FactValue</code> , <code>.sync</code> or <code>.awaitable</code>	Declares named <code>ActionParams</code> with a required flag; description starts with <code>[ACTION]</code> by convention

Each entry also carries an `availability` closure (3.1.4). Factory functions default availability to "always available" so a declaration is one expression.

One line, one fact. A namespace resolver exposes only `namespace: String` and `entries: [FactEntry]`. The same `entries` array serves two purposes: the registry derives the model-facing tool schema from it, and the

default resolve walks it to dispatch a key. The resolver closure is a field of the entry, so there is no second table that could drift from the first. A fact that can be resolved is a fact that is published, and the reverse.

Registry contents at e028039. 26 namespace resolvers are registered: 16 for the user's health data, 9 for app/device state and action surfaces, and one composites namespace registered last. Some share a head token: five register as `app` and two as `workout`. The source declares **238 entries**: 149 `fixed` with synchronous bodies, 8 `fixed` with awaitable bodies, 56 `parameterized`, 6 `composite` and 19 `action`.

3.1.1 Key grammar

Model and resolver use the same key grammar, so no translation layer sits between them:

```
key      := token ( "." token ) *
token    := name | name "(" argument ")"
```

Parsing steps:

1. Trim whitespace; reject empty input.
2. Scan characters, tracking parenthesis depth. A `.` at depth 0 ends a token. A `)` that would make the depth negative is an error. A non-zero depth at the end is an error. An empty token (`a..b`, leading dot) is an error.
3. A token whose last character is `)` and which contains `(` splits into name and argument. An empty name is an error.

Dots inside arguments are therefore preserved (`session.by_date(2026.04.21)` is valid).

Pattern matching. The pattern is parsed with the same grammar. For each pattern token in turn, the names must be equal. If the pattern argument begins with `$`, the key's argument is captured, and a key with no argument fails. Otherwise both arguments must be equal or both absent. The pattern must have at most as many tokens as the key. Remaining key tokens become `tail`. At least one capture must occur.

Dispatch. The registry checks first every namespace whose `namespace` equals the key's head token, in registration order. Several namespaces may share a head. It then scans all other namespaces, which is how composites stored in `composites` resolve under keys such as `user.profile.snapshot`. If nothing matches, it returns `missing(notRecorded, "no such key")`.

Tool name derivation. Dots become underscores. A `($param)` section is removed and the parameter becomes a named string property on the tool: `session.by_date($date)` becomes the tool `session_by_date` with required string `date`.

3.1.2 The closed value union and envelope

`FactValue` has nine cases: `integer`, `double`, `string`, `date`, `durationSec`, `boolean`, `missing(reason, detail?)`, `list`, `record`. Every tool result is serialised into one envelope with sorted keys:

```
{ "value": <typed JSON or null>,
  "missingReason": <reason, present only when absent>,
  "detail": <optional hint>,
  "asOf": <ISO-8601 with local UTC offset>,
  "confidence": "high" }
```

Serialisation rules: a non-finite double becomes `null` for that field only. Dates use the device's local offset, so the calendar day matches the `YYYY-MM-DD` arguments the tools accept. A nested `missing` inside a list or record is rendered as an inline envelope, so absence can be reported per field. On a successful result `missingReason` is omitted entirely. The presence of the key is the absence signal. The cloud tool loop and the on-device dispatcher for Apple's framework both call this serialiser, so all six providers receive the same shape.

What the envelope does not carry. `asOf` is the time of resolution, not the time of measurement, and `confidence` is the constant `"high"`. Neither is provenance. The real provenance signals are fields inside specific records:

- `data_source`: `sport_wide` versus `route_specific` for cross-workout baselines, or `live_healthkit_pending_acceptance` for sleep and vitals read live before the night is accepted. Each tool's description tells the model how to word comparisons for that source ("compared to your typical run" versus "on this loop").
- `comparison_safe`: `false` when fewer than 2 prior samples back a baseline.
- Composite attribution (3.1.5): which children are present and which are missing, and why.

3.1.3 The absence taxonomy

Every resolver that cannot return a value must pick one of nine reasons. There is no free-text-only absence.

Reason	Meaning to the model
<code>notRecorded</code>	No data for this query (most common)
<code>notYetComputed</code>	Data exists; the derived metric is not computed yet. Retry later
<code>outOfRange</code>	Parameter outside the fact's valid range
<code>sensorDropout</code>	Signal too noisy or sparse to trust
<code>invalidParameter</code>	Malformed arguments; <code>detail</code> names the parameter and accepted range
<code>internalError</code>	Bug or budget overrun; report unavailable and stop
<code>tooMuchData</code>	Result over the size cap; narrow the query
<code>rateLimited</code>	Same call missed repeatedly this turn, or the turn's tool budget is spent
<code>partialData</code>	Composite: some children missing (see 3.1.5 for how the code actually signals this)

3.1.4 No-I/O availability gating

Each entry's `availability` closure returns `{hasData, validRange?, lastUpdated?}`. It must be synchronous and metadata-only: no HealthKit, file or network access. In practice it reads in-memory archive indexes and settings flags. The schema builder runs it once per build:

1. Drop every entry with `hasData == false`. The per-entry schema then never offers a tool for data the user does not have.
2. For a parameterised entry with a `validRange`, append to the parameter description: "Example: `<example>`. Data available as early as `<Month YYYY>`; do not request earlier dates.". Only the month of the lower bound is given, with no rolling end date, so the text changes at most once a month per entry.

The workout-backed availability, for example, is `unavailable` until the archive holds one workout. After that it is valid from the earliest to the latest archived workout date.

3.1.5 Composites with declared dependencies

A composite lists the atomic keys it reads. Composites may not depend on other composites. A test validates that every dependency names a real entry. On the asynchronous tool path:

1. Capture the composite's parameter (if parameterised) and substitute it for any `$name` in each dependency string.
2. Await each distinct dependency through the async resolver. This is where HealthKit or network children suspend.
3. Merge the results into a registry-held prefetch map, run the synchronous composite body (whose `registry.resolve(key)` calls hit the map first), and then restore the previous map. Nothing suspends between filling and restoring the map, so no other resolve sees it.
4. The body builds `{present: [{key, value}], missing: [{key, reason, detail?}]}` and adds `status: "partialData"` when any child is missing.

A synchronous walk that reaches an awaitable child it did not prefetch returns `missing(internalError, "resolves asynchronously ...")`. It neither blocks nor traps.

3.1.6 Registry rails

Every tool result passes through a final gate in this order:

1. **Repeat-miss rate limit.** The signature is `toolName + "::" + argsJSON`. Each *missing* result increments a per-turn counter for that signature. When the counter exceeds 2, the result is replaced by `missing(rateLimited, "Same tool+args returned missing 2 times this turn. Stop retrying...")`. The first two identical misses pass, and the third is short-circuited. Successful results do not count. Counters reset at the start of each user turn.
2. **Size cap.** A non-missing result whose serialised envelope exceeds 80,000 UTF-8 bytes is replaced by `missing(tooMuchData, "Response ~N bytes exceeds 80000-byte per-call cap. Ask a narrower query...")`.
3. **Time budget.** On the per-entry tool path, if resolution took more than 2.0 s of wall-clock time, the value is discarded and `missing(internalError, "resolver timed out...")` is returned. Over 0.5 s logs a warning. Over 4.0 s on an action logs a watchdog warning. On the compact read-tool path the budget only logs, because legitimate HealthKit and network waits can exceed 2 s.

4. **Numeric argument range checks.** Every numeric parameter is declared once with a closed range, an optional excluded lower bound and an integer flag. Parsing accepts only a finite number inside the range, and a whole number where the flag is set. Anything else, including "inf", "nan" and "1e20", returns `invalidParameter` with the requirement in words, e.g. "ordinal must be a whole number from 0 to 100000, got '-1'". Comma-separated parameter blobs must have exactly the expected field count, and empty fields are counted, so `60,,5` has three fields and is rejected.
5. **Required action arguments.** A missing required argument returns `invalidParameter` naming it, before the action body runs. Unknown argument keys are dropped.

Catalogue hash. The availability-filtered per-entry schema is JSON-encoded with sorted keys and hashed with SHA-256. The hash is recomputed on each send to detect drift in cache-relevant bytes. It changes only when an entry, a description or an availability result changes.

3.2 Compact tool layer

The model sees 39 tools, not 238:

- **21 read tools** with fixed names and string parameters. Most take a selector (`which`, `field`, `metric` or `aspect`) and route to one or more catalogue keys: `get_today`, `get_session`, `get_recovery`, `get_hrv`, `get_sleep`, `get_vitals`, `get_workout`, `list_workouts`, `get_walks`, `get_baseline`, `get_user`, `get_app_state`, `get_training_load`, `get_score_meta`, `get_breadcrumbs`, `get_routes`, `get_tags`, `get_workout_live`, `get_assistant_state`, `get_healthkit`, plus `lookup_fact`, which resolves any raw catalogue key.
- **18 action tools** keep their catalogue names (e.g. `assistant_email_compose`, `routes_library_rename`, `web_search`, `directions_routeTo`) so that each side effect stays identifiable in audit logs. Their specifications are taken from the availability-filtered per-entry schema and filtered by an explicit allow-list. The catalogue declares 19 actions. `location.roads_ahead` is not on the allow-list.

Construction: read tools are appended first, then actions. Duplicates by name are dropped, first occurrence wins, and a warning is logged, because one provider rejects duplicate tool names. The list is then sorted by name. Action input schemas list `required` sorted. Every property is a string, and the resolver converts types.

Capability index. The read-tool list also generates a short system-prompt section. For each tool it prints the name and the description's first sentence, capped at 170 characters. The section tells the model to call the matching tool before answering any question about the user's own data. Because it comes from the same specification list, it cannot drift from the real schema, and it is byte-stable.

Retrieval and caps. A BM25 ranker [13] ($k_1 = 1.5$, $b = 0.75$, target 40) runs over the compact schema. At 39 tools it returns the list unchanged. The on-device provider, whose context holds only a few tools, uses the BM25 order to choose which to keep. A per-provider cap (110 for one provider) truncates the ranked list.

Per-turn call budget. The loop counts tool calls across rounds. When `callsSoFar + callsThisRound > 8`, every call in that round gets `missing(rateLimited, "tool budget exceeded for this turn")`. The model then has one more round to write a text answer, and any further tool calls are dropped. The on-device dispatcher enforces the same constant.

3.3 Runtime number verification

Scope. The verifier checks only claims about the value *now*. Two families exist:

Family	Metrics	Source of truth	Where applied
Live workout	HR, power, DFA α 1, HR drift	Live workout snapshot	Voice, each chunk before speech
App state	TSB, ATL, CTL, ACWR, RMSSD, recovery score, last night's sleep duration and efficiency	Training-load registry (same accessor as dashboard and tools); latest reliable overnight session; latest overnight with sleep	Chat, every tool-loop round end; voice, each chunk before speech

Table 2. Claim shapes and tolerances

Metric	Claim shape (case-insensitive, simplified)	Tolerance	Replacement format
HR	"your [current/live] HR/heart rate [is/s]..." or "HR right now/currently/now", then filler words, then $\backslash d\{2,3\}$ bpm	5 bpm	N bpm
Power	same shape, "power/wattage", $\backslash d\{2,4\}$ W/watts	15 W	N W
DFA α 1	same shape, " α 1/alpha 1", optional "DFA", decimal only	0.10	%.2f
HR drift	same shape, "[HR] drift", number + %	1.5 points	%.1f%
TSB	TSB + optional "negative" + signed number	2.0	%+.1f
ATL / CTL	label + number	3.0	%.1f
ACWR	label + number	0.15	%.2f
RMSSD	RMSSD [of] N [ms]	8.0 ms	%.0f ms
Recovery	recovery N	4.0 on 0–100; a claim ≤ 10 within 1.0 of the 0–10 score is accepted	on the claim's scale
Sleep duration	"slept [for] [about] Nh [M m]", "sleep [duration] was ...", "... of sleep"	5 min if minutes or decimal hours given; 30 min for whole hours	"6h 55m" or "6 hours and 55 minutes", matching the claim
Sleep efficiency	"[sleep] efficiency [of/was/...] N%", "N% [sleep] efficient/efficiency", only in a sentence that mentions sleep	1.0 point	N%

Each app-state pattern rejects a number followed by a unit of time or count (days, h, weeks, nights, minutes, sessions, workouts), by %, or by more digits. "CTL 42-day window" is therefore not a claim.

Algorithm (app-state family).

1. **Normalise spelled numbers.** Match $\backslash b(negative\backslash s+)?(ones-or-teen | tens([\backslash s]+digit)?)(\backslash s+point\backslash s+digit)?\backslash b$, with alternations sorted longest first. Replace each match with digits ("negative

thirty-one point seven" → -31.7), keeping a list of (originalRange, normalizedRange) rewrites. Range: 0–99 with at most one decimal digit, English words only.

2. Run the metric regexes on the normalised text. Each regex names two groups: **value** (the number) and **claim** (number plus unit, the span to replace). The metric label is never part of the replaced span.
3. For each match, find its sentence using a sentence enumerator that does not split "17.2". Discard the match if the sentence contains a **time marker** (ago, yesterday, last week/month/.../weekday, previous(ly), earlier, before, on) or a **target/comparison marker** (average, mean, baseline, typical, usual, target, goal, keep, stay, peak, max, min, trend, would, could, if, should, threshold, under, below, above, over, between, range, higher, lower, than, and similar).
4. If the claim reads "negative N", negate the value. Compute $|\text{claimed} - \text{actual}|$ and keep the match only if it exceeds the tolerance.
5. Recovery: if $|\text{claimed} - \text{score}_{10}| \leq 1.0$, drop the match (same score on the 0–10 scale). If the claim is ≤ 10 , report it on the 0–10 scale.
6. Sleep: drop matches in a sentence with general-norm markers ("need", "recommended", "adults", "per night", ranges such as "7-9") or other-night markers (month and weekday names, "nights", "weeks", dates). Drop overlaps.
7. **Remap** each range from normalised to original text. An offset inside a rewritten span snaps to the rewrite's original start or end, so "negative thirty-one point seven" is replaced whole.
8. Build **correctedText** by replacing claimed spans back-to-front. Every other word, including unrelated spelled numbers ("two sessions"), is unchanged.

The live-workout family follows steps 2–4 without normalisation and requires the "your X is" / "X right now" shape. A bare "150 bpm" is usually a target or a different metric.

Application. In chat, verification runs at the end of every tool-loop round, including rounds that end in a tool call, before the pre-tool text is rewound. Text published during that round may already have been visible. The corrected text replaces the saved turn. In voice, each chunk passes output scrub → live-workout verification → app-state verification → preamble → speech normalisation, so a corrected number is the only one spoken.

Closed correction loop. Every discrepancy is added to a process-wide, lock-guarded buffer. Entries with the same (metric, claimed, actual) are not re-added, because voice catches the same claim in the spoken chunk and again in the saved turn. Only the 4 most recent are kept. When the next system prompt is composed, the buffer is read and cleared into a variable section:

```
# Last turn correction – DO NOT FABRICATE
Your previous response contained numbers that contradicted live data. The saved reply was
corrected, but the user may already have read or heard the wrong number – if it matters,
correct it briefly. For any of these metrics next turn, CALL the appropriate tool (...)
and
quote what comes back – never estimate or interpolate.
• You said TSB=-29.0 – actual was -15.5.
```

One fabrication therefore produces exactly one reminder on the very next turn. The section sits outside the cached prompt prefix.

3.4 Structural resolution of user-asserted values

Before the variable prompt sections are built, a detector reads the last 6 conversation turns. It keeps only user turns that are not `localOnly`, and walks them newest first. For each message:

1. Normalise spelled numbers (3.3 step 1).
2. Remove every sentence ending in `?` or `!`. Questions assert nothing.
3. For each known metric name (training load, HRV, heart rate, recovery, sleep, workout and vitals terms; longest first so "max hr 190" is not read as "hr 190"), find the *last* `\bmetric\b\s*[connector]\s*number` in the remaining text. The connector is optional except for `sleep` and `recovery`, which need one of `is/of/=/:/was/being`. The number may not be a clock time (5 am, 11 pm, 5:30, o'clock). Each match is blanked after it is used. The newest assertion per metric wins. Transcription variants (`bsb`, `tsp`, `tbs` → `tsb`, and so on) fold into the canonical key only if the canonical spelling gave no value.
4. Set `dashboardContradicted` if the text contains a contradiction phrase ("the dashboard says", "real numbers are", "you're wrong", "wrong number", ...) or a sentence opening "no/actually, it's/it is" followed by a number or a word such as not/different/wrong/higher/lower/more/less.
5. Set `explicitOverrideRequested` on phrases such as "stop calling tools", "use these numbers", "I don't care what the dashboard...", "stop saying".

Policy, applied in the composer:

- If any signal is present, inject `# User-stated values (authoritative – fresher than any cache)` with one `- key: value` line per assertion (training-load metrics first, fixed formats). If a contradiction or override was seen, add a sentence saying the cached data is stale.
- If `dashboardContradicted` or `explicitOverrideRequested`, **omit the dashboard training-load block entirely**. The model then has no stale number to reconcile.

3.5 The multilingual regulated-claim lexicon

Concept structure. A concept has an `id`, a list of Latin-script alternatives and a list of "unbounded" alternatives. Its regex is

```
(?: \b(?:latin_1|latin_2|...)\b | (?:unbounded_1|unbounded_2|...) )
```

compiled case-insensitively against the original text, never a case-folded or diacritic-folded copy, so match ranges stay aligned with the string being rewritten. Accented forms are spelled out (`m[ée]decin`). Word boundaries apply only to Latin-script alternatives. CJK text has no inter-word boundary, so a `\b` around `心房細動` never matches in running Japanese. CJK and Arabic alternatives are therefore matched as substrings.

Languages. English plus `ar`, `da`, `de`, `es`, `fi`, `fr`, `is`, `it`, `ja`, `ko`, `nb`, `nl`, `pt-BR`, `ru`, `sv` and `zh-Hans`: 17 in all. A test fails if any concept has no non-Latin alternatives.

Concepts (29). Rhythm: atrial fibrillation, arrhythmia, irregular heartbeat. Conditions: named cardiac condition, acute cardiac event, neurovascular event. Claims: diagnosis, cure, treatment claim, prescription, regulatory clearance, pathology, clinical physiology labels, symptom of disease, speculative diagnosis. Risk framing: injury risk, overtraining, risk-zone framing. Certainty: categorical autonomic state, physiological certainty, unsupported

metric verdict. Referral: medical referral. Input-side: chest pain, breathlessness, syncope, severe symptom, danger judgement, self-harm, injury. A separate two-concept offensive-language list uses the same structure.

Benign-homonym exclusions. These are written per alternative because the qualifier's position differs by language:

Benign sense	Exclusion
Respiratory sinus arrhythmia (normal physiology)	EN (?<!sinus\s)arrhythmias?; FR arhythmies?(?!s+sinusales?); ES/PT arritmias?(?!s+sinusal(?:es)?); NL/DE/DA/NB/FI (?<!sinus)_; RU (?<!синусовая\s)(?!синусовой\s)аритми[яи]; JA (?<!洞性)不整脈; ZH (?<!窦性)心律失常; KO (?<!동성)(?!동성)부정맥; AR negative lookahead for / التنفسي الجيني . German Sinusarrhythmie escapes because the compound has no inner \b
Software diagnostics ("Diagnostic log")	`diagnos...(?!s+(?:log
Swimming "stroke rate", "stroke of luck"	no bare stroke ; only `(?;having
"Suicide sprints" (a drill)	`su[ɪi]cide(?:s+(?:sprints?
Finnish <i>parantaa</i> ("improve"), Italian <i>prenditi cura</i> ("take care"), Icelandic <i>greining</i> ("analysis"), Germanic <i>Rezept/recept</i> ("recipe")	only disease-object or prescription-only shapes match
"This is your recovery score"	(?-i:[Tt]his\s+IS\s+your) is case-sensitive inside a case-insensitive pattern, so only the emphatic claim matches

Groups. Three named groups select concepts, each for a stated reason:

- **refuseBeforeSending** (12) = rhythm (3) + emergency (chest pain, breathlessness, syncope, severe symptom, self-harm, acute cardiac event, neurovascular event) + general concern (danger judgement) + injury.
- **scrubFromOutput** (21): all rhythm, condition, claim, risk, certainty and referral concepts.
- **neverInStaticCopy** (12): rhythm (2), condition (3), regulatory clearance, cure, prescription, treatment, speculative diagnosis, pathology, clinical physiology labels.

Eight concepts are **input-only**. Chest pain, breathlessness, syncope, severe symptom, self-harm and injury are input-only because the model must be able to say these words when explaining why the app cannot help, and scrubbing them would rewrite the safety copy. Danger judgement consists of bare adjectives (**dangerous**, **gefährlich**) that the app's own educational copy uses in negating constructions ("not because the number is dangerous in itself"); assertion-shaped danger framing is covered on output by risk-zone framing. Acute cardiac event is a strict subset of named cardiac condition, which is already scrubbed. It exists so that the input guard can refuse "am I having a heart attack?" without refusing "is my resting HR bradycardia?", which is an ordinary question from an endurance athlete. Every one of the 29 concepts is in exactly one of these two roles, or in both (the rhythm and neurovascular concepts).

The `neverInStaticCopy` group excludes concepts the app's own copy uses on purpose. The localized safety redirect "Consult a doctor if consistently low" matches `medicalReferral`, and running the whole lexicon over the catalogue produced 45 such matches. The 12 selected concepts matched nothing in any locale when measured, so the group can act as a hard gate.

3.6 Input guard

Classification order. The order is a safety decision. The first group that matches wins:

1. Self-harm → crisis reply
2. Emergency group → symptom reply
3. Injury → injury reply
4. Rhythm → rhythm reply
5. Danger judgement → symptom reply
6. none → proceed

Justification for the order: an acute symptom outranks a rhythm question because its reply names an emergency number. "Skipped beats and chest tightness" must not get the reply that recommends a smartwatch ECG. A request for a risk judgement ranks *below* rhythm because "should I be worried about my AFib" is better served by the ECG pointer. Self-harm comes first because it was once classed as an emergency and got "talk to your doctor" with no crisis line.

Replies. Each reply is a fixed string from the string catalogue, so it is localized into all 17 languages:

- Crisis: an apology, "please reach out to someone now", the local emergency number, findahelpline.com, and 988 for the US. It does not mention a doctor.
- Symptom: "If you're exercising, stop now. Talk to your doctor about that. I can't assess your health — Emuqu is a fitness coaching app, not a medical device. If you're feeling unwell, please contact a clinician (or your local emergency number for severe symptoms)."
- Injury: stop, call the local emergency number if serious, the app cannot assess injuries, otherwise see a clinician.
- Rhythm: the app does not detect AFib or arrhythmia and is not a medical device; talk to your doctor; a clinically validated ECG feature exists for that purpose.

Effects on a refusal.

1. Append the user turn and the canned assistant turn, both flagged `localOnly`, and persist them.
2. Return without calling any provider, including the on-device one, and before the consent check.
3. The `localOnly` flag lives on the stored turn, not at one send site. History assembly withholds such turns before budgeting, so they are never counted as "dropped". Summarisation filters them again, and the correction detector skips them. A guarded message therefore cannot reach a provider on a later turn as history.

Feature flag. The guard is behind a flag that defaults on. The self-harm branch runs even when the flag is off: `if flagOn || classify(text) == selfHarm.`

Prompt parity. The system prompt's medical-boundary rule B asks the model to reply exactly with the symptom, self-harm and injury texts. Unit tests check that the symptom and injury replies appear verbatim in both prompt variants (cloud and on-device), after whitespace is collapsed. A user gets the same words whether the guard or the model handles the message.

Voice echo trap. Voice speaks the refusal aloud, and the rhythm refusal itself contains "AFib" and "arrhythmia". A transcript is therefore dropped as an echo when (a) it would trigger the same refusal that is already the last assistant turn and (b) at least 40% of its tokens overlap that reply.

3.7 Output guard

Scope. The output guard deflects sentences that name a condition for the user, speculate about one, assess rhythm, claim cure, treatment, prescription or regulatory clearance, frame risk as an injury prediction or "danger zone", assert categorical autonomic certainty, or rank a metric as truth. General physiology passes untouched: HRV, parasympathetic activity, respiratory sinus arrhythmia, training load, sleep. The prompt (rules A and D) also tells the model not to give dosing or personal prescriptions. On output, only prescription vocabulary is matched. There is no separate dosing regex.

Negation-neutral by design. The guard does not try to detect negation. A denial is a finding too: "This isn't AFib" rules a condition out. Negation is also spread across a sentence in ways one pattern cannot follow in 17 languages (sentence-final in Japanese and Korean, split in French, a negative verb in Finnish). Each deflection is therefore written to be true whether the replaced sentence affirmed or denied the term. For example: "Emuqu measures beat-to-beat timing; it doesn't assess heart rhythm. A clinician or a clinically validated ECG is the right place for that question." Deflections are catalogue strings and follow the in-app language. A test checks that no deflection trips any rule. The output guard has 13 distinct medical deflections: 9 specific to one concept, plus 4 shared by a group of concepts (rhythm, load, out-of-scope, and an observation fallback). One further line replaces offensive language.

Scrub algorithm.

```

scrub(text):
  active = medicalRules + offensiveRule (+ englishOnlyRule if it matches and reply is
  English)
  if text empty or firstMatch(text, active) is none: return text unchanged
  segments = split(text) so that concat(segments) == text exactly
  for each segment:
    core = segment without leading/trailing whitespace
    rule = firstMatch(core, active)
    if rule: emit leadingWS + rule.deflection + trailingWS; record (rule.reason, core)
    else:    emit segment byte-identical
  if nothing was replaced:      # match straddles a terminator
    m = rule.regex.firstMatch(original text)
    rebuilt = text with every segment overlapping m collapsed into ONE deflection
    if m found and firstMatch(rebuilt, active) is none: return tidy(rebuilt)
    else: return rule.deflection # whole message
  return tidy(rebuilt)          # " "->" ", " ."->".", " ,"->","

firstMatch(t, rules):
  r = first rule whose regex matches t
  if r: return r
  plain = t with every '*', '_', '`' removed
  return plain != t ? first rule matching plain : none

```

Segmentation: a segment runs to and including a terminator from `{. ! ? \n . ! ? , ? ...}`, then any run of further terminators (`?! , ...`), then whitespace. A `.` after a digit is a decimal point when a digit follows. When the text ends right after it, it is undecided and does not split. A sentence is rewritten at most once, and rewritten text is never re-scanned. The pass is linear and needs no iteration cap. An earlier design used a 32-replacement cap, which let 136 of 200 prohibited sentences through.

The markdown retry exists because `You may have *atrial* fibrillation.` evades `atrial\s+fib`. It is safe because callers that need a range re-run the regex on the original text and fail closed.

Where it runs.

1. **Streaming buffer (chat and the voice cursor).** Deltas accumulate in a buffer that publishes at most every 33 ms. Each publish splits the buffer at its last terminator. Only the complete part is scrubbed and appended to the visible turn. The incomplete tail is held back, so a prohibited phrase never renders, even partially. The voice "speakable cursor" advances only over published text. At the end of a round, and on every exit path including errors and Stop, the tail is force-published, scrubbed.
2. **Speech.** Every utterance passes through one function before the synthesiser: `scrub` → number verification → preamble. That covers streamed chunks, completed turns, error lines and wake-word acknowledgements.
3. **Final pass** over the whole message after the stream ends, for text that did not go through the buffer.

3.8 Build-time perimeter

Copy linter. The linter scans the whole app source tree, the Watch app and both string catalogues. It has three properties beyond plain pattern matching:

1. **A real Swift literal scanner**, not a quote-pairing regex. It walks the source once, handling `//` comments, *nested* `/* */` comments, `"""` multi-line literals, and raw literals (`#"..."#`, `##"..."##`), where backslash

is not an escape. Interpolations `\(expr)` are blanked before matching. A quote-pairing regex, measured on this code base, missed 93% of the characters inside multi-line literals, including all long-form help articles.

2. **Prohibition-context exemption.** The system prompt quotes forbidden framings in order to forbid them ("Never tell the user that a number 'predicts injury'..."). A match is excused only when (a) a guard pattern such as `never|don't|do not + tell|say|use|describe|call|label|refer|frame|claim|imply` matches inside a 3-line window of the **comment-stripped** source and the first 40 characters of the literal (at least 3) lie inside that same sentence, or (b) the sentence containing the match, inside the literal, contains a guard. A comment cannot excuse a string. String-catalogue values are never excused this way.
3. **Catalogue handling.** An allow-listed catalogue *key* also clears all its translations, so the wellness disclaimer does not fire once per locale. Two pattern sets run over every catalogue: the English build-time list (62 patterns) and, for *every locale*, the multilingual `neverInStaticCopy` concepts. The linter does not copy that second set into its configuration. It parses it out of the Swift lexicon source, taking each concept's `latin` and `unbounded` arrays with bracket matching, unescaping one level, and compiling them with the same `\b` rules as the runtime.

Fail-closed rules: a missing scan path, an unreadable file, an unparseable catalogue, a missing `neverInStaticCopy` group or zero usable concepts, or a pattern that fails to compile each exit with code 2.

Coverage gate. The gate proves the runtime lexicon matches every build-time term. It runs in CI:

1. Extract every literal from every `latin:/unbounded:` array in all lexicon files (a glob, so a file split does not hide terms). Bracket matching skips string literals, so a `]` inside a character class does not close the array. Compile each one, word-bounding Latin alternatives.
2. **Fail (exit 2)** if any alternative matches the empty string, or if fewer than 100 alternatives were extracted. Either means the parser is broken and the gate would pass vacuously. At e028039 there are 985.
3. For each build-time pattern, **synthesise a witness string**: remove `(?i)` and lookarounds; replace `\s+` with a space; remove `\s*` and `\b`; reduce character classes to one member (`\s` → space, `\x` → `x`); drop leftover quantifiers; repeatedly reduce `(a|b)?` → `""`, `(a|b)` → `a`, `(a)?` → `""`, `(a)` → `a`; remove `?`.
4. **Fail (exit 2)** if the witness does not match its own source pattern. The synthesiser would then be wrong.
5. **Fail (exit 1)** if no lexicon alternative matches the witness, listing each gap.

At e028039: 62 patterns probed against 985 alternatives, clean.

Exported reports. PDF reports and the post-workout Coach Report email body are built from templates, not from model output. They contain no LLM text. Their wording is covered only by the build-time linter. The runtime guards do not see them.

3.9 Consent and provider abstraction (context, mostly not claimed)

Each hosted provider needs its own consent before its first send. The consent key includes a schema version (currently 8) that is bumped when the disclosure changes, and removing an API key revokes consent. The on-device provider is exempt. When a provider fails, the fallback chain contains only other enabled, available, consented hosted providers, then the on-device provider. When the on-device framework refuses a request on safety grounds, the request is shown as refused and not retried on another provider. Each outbound request is recorded before provider-specific transforms in an in-memory first-in-first-out audit of 10 entries.

4. What Is New

N1. A fact catalogue in which each fact is a single declarative entry that binds key or key pattern, description, value type, a no-I/O availability check and the resolver closure, and from which both the model-facing tool schema and the key dispatch are derived, so that no resolver can exist without being published to the model and no published tool can lack a resolver.

N2. A shared key grammar (dotted tokens with parenthesised arguments, depth-aware splitting) used verbatim by the model, the pattern matcher and the tool-name derivation, with leading-token pattern matching that passes trailing tokens to the resolver as a sub-key.

N3. A uniform tool-result envelope with a closed nine-value absence taxonomy, delivered identically to five hosted providers and one on-device model, in which the presence of the absence key is itself the absence signal and nested absences are reported per field.

N4. A registry gate that applies, in order, a per-turn miss counter on the exact (tool, arguments) signature that short-circuits the third identical missing result, a byte cap that converts oversized results into a "narrow the query" absence, a wall-clock budget that discards late values, and declared numeric parameter ranges that turn malformed model arguments into absences naming the accepted range.

N5. Composite facts that declare their atomic dependencies, have those dependencies awaited and placed in a scoped prefetch map before a synchronous composite body runs, and return a present/missing attribution record with per-child absence reasons.

N6. A two-tier tool surface of fixed polymorphic read tools plus allow-listed action tools taken from the availability-filtered catalogue under their original names, de-duplicated and name-sorted for byte stability, with a system-prompt capability index generated from the same read-tool specification list.

N7. A runtime verifier that checks only numeric claims about a metric's current value, excluding by sentence-level time, target and comparison markers, and that compares each claim against the app's own source of truth with per-metric tolerances after normalising spelled-out numbers and mapping match ranges back to the original text, so that only the claimed number-and-unit span is replaced in the chat transcript and in each voice chunk before it is spoken.

N8. A closed correction loop in which verified discrepancies are de-duplicated by (metric, claimed, actual), capped at four, and consumed exactly once into the next turn's system prompt with an instruction to call the named tool.

N9. Structural resolution of conflicts between user-stated values and cached dashboard values: detecting statements (not questions) that assert metric values or contradict the cache in recent non-local user turns, then removing the cached block from the prompt and injecting a user-stated block, instead of instructing the model to prefer the user.

N10. A single regulated-claim lexicon of concepts in seventeen languages, each split into word-bounded Latin-script alternatives and substring-matched CJK/Arabic alternatives, with per-language, position-aware lookarounds for benign homonyms, consumed through named groups (input refusal, output scrubbing, never-in-static-copy) whose membership is justified per concept.

N11. An input guard that classifies the current message in a fixed safety order (self-harm, acute emergency, injury, rhythm, danger judgement), answers with a fixed, catalogue-localized reply that unit tests show the system prompt quotes word for word, persists both turns with a local-only flag that every outbound path honours, sends nothing to any provider, and answers self-harm even when the guard's feature flag is off.

N12. A negation-neutral output guard that replaces whole sentences with deflections written to be true whether the sentence affirmed or denied the term, retries on markdown-stripped text, deflects the whole message when a match straddles a sentence boundary and cannot be localised, and runs on complete sentences before they are displayed (holding incomplete, decimal-aware tails in the streaming buffer), on every spoken utterance, and in a final pass.

N13. A build-time copy linter with a real Swift literal scanner, a prohibition-context exemption evaluated only on comment-stripped source and only within the prohibiting sentence, key-level allow-listing that covers translations, and multilingual patterns parsed from the runtime lexicon and applied to every locale of the string catalogues, failing closed on any missing input.

N14. A coverage gate that synthesises a witness string for each build-time regex, verifies the witness against its own pattern, and requires the runtime lexicon to match it, and that refuses to pass when any lexicon alternative matches the empty string or fewer than one hundred alternatives were extracted.

Not claimed (prior art). Model tool or function calling [3][4][14]; retrieval-augmented generation [2]; provider-side prompt caching [15]; BM25 ranking [13]; regular-expression keyword filtering in general; classifier- or rail-based LLM guardrails [10][11]; post-hoc claim verification with model calls [5][6]; clinical negation detection [12]; multi-provider fallback; per-vendor consent prompts; instruction hierarchies [9][16].

5. Accepted Tradeoffs and Limitations

Validation status. The science register (`Tools/science_register/register.json`, scoring version v3.1.oct2026) holds 19 entries: 3 validated, 4 supported-transfer, 12 awaiting-validation. **None covers any method in this paper.** These are software controls, not physiological heuristics, and none is described here as validated. The verifier tolerances are hand-set from dashboard rounding and the noise of each metric. They have unit tests but no measured false-positive or false-negative rates.

Lexical, not semantic. Every perimeter element is a regular expression. A paraphrase that avoids the vocabulary passes, and the build-time list has already been evaded once ("strongly suggests" for "strong predictor"). Specific known gaps:

- **Obfuscation.** There is no normalisation of leetspeak, homoglyphs or inserted characters beyond removing `*`, `_` and backticks. The offensive list handles a few variants (`n[i1]gg...`); the medical lexicon does not.
- **Single-message input guard.** The input guard sees only the current message. A question split across turns ("I have a heart condition" ... "is it atrial...") is not joined.
- **Auto-extracted memories** are written from the user and assistant texts by a background model call. They are not passed through the lexicon, and they are injected into later system prompts.
- **Model-drafted email bodies** (`assistant_email_compose`) go to the system mail composer without passing through the output guard. The user reviews the draft and sends it themselves.

- **English-only number verification.** The claim regexes and the spelled-number normaliser (0–99, one decimal digit) are English. A wrong number in a German reply is not corrected.
- **Over-deflection.** Negation-neutrality means a correct denial ("there's no sign of overtraining") is also deflected. A sentence containing a matched term anywhere loses the whole sentence.
- **Prompt and guard disagree.** Prompt rules A and C allow the model to describe a disease or AFib in general terms. The input guard refuses any message naming AFib, and the output guard deflects any sentence naming a cardiac condition. The guards win, so general information about named conditions is not delivered.

Verification scope. Only "value now" claims for the 11 metrics in Table 2 are checked. History, averages and targets are skipped by design, which also means a wrong historical number passes. Live-workout claims (HR, power, $\alpha 1$, drift) are checked only on the voice path. Chat checks app-state claims only. In chat the correction is applied at the end of the round, after the round's text has been displayed. The next-turn reminder tells the model the user may have seen the wrong number.

Catalogue honesty.

- `asOf` is resolve time and `confidence` is always "high". Neither carries information.
- The `partialData` reason exists in the enum, but composites report partial results as a successful record with `status: "partialData"`. No resolver returns it as the top-level reason.
- Availability gating filters the per-entry schema, which feeds the action-tool list and the catalogue hash. The 21 compact read tools are static, so they are not removed when data is absent, and the valid-range text written into per-entry parameter descriptions is not shown to the model in the compact configuration.
- The `[ACTION]` prefix is a convention, not an enforced check. One allow-listed action (`directions_routeTo`) lacks it. Several location *reads* are declared as actions.
- When a round would exceed the 8-call budget, every call in that round is refused, including calls that would have fitted.
- The 2 s time budget discards values only on the per-entry path. Compact read tools log but keep slow results.

Build-time gates.

- The coverage gate is one-directional: build-time terms must be covered at runtime, not the reverse. It accepts a match by *any* lexicon alternative, including input-only concepts that the output guard does not scrub. It skips a pattern whose witness reduces to an empty string.
- The linter's multilingual extraction reads only the main lexicon file and silently skips an individual concept it cannot locate or compile. It fails closed only when no concept is usable.
- Exported PDF reports contain no model text and are covered only by the build-time linter.
- The consent-schema bump depends on discipline. No automated check exists.

6. Comparison to Existing Approaches

Approach	Grounding of numbers	Output check timing	Languages	On-device, per sentence	Build-time copy link	Source
Function calling / tool use (OpenAI, Anthropic)	Model may call tools; output not checked	None	Any	No	No	[14][15]
RAG	Retrieved passages in context; output not checked	None	Any	No	No	[2]
RARR / FActScore	Post-hoc claim check with model calls and retrieval	After generation	Primarily English evaluated	No	No	[5][6]
NeMo Guardrails	Programmable input/output rails; can call a model	Configurable	Configurable	Not documented for on-device streaming	No	[11]
Llama Guard	Classifier over prompt and response safety categories	After generation (model call)	Primarily English in original paper	No	No	[10]
NegEx-style rules	—	Batch	English (original)	n/a	No	[12]
LLM coaching features in consumer wearables (e.g. WHOOP Coach, Oura Advisor)	Not publicly documented	Not publicly documented	Not publicly documented	Not publicly documented	Not publicly documented	—
Emuqu (this paper)	Typed catalogue + "value now" verifier with per-metric tolerance and span correction; next-turn feedback	Before display (complete sentences) and before speech	17, one lexicon	Yes, regex, no model call	Yes, same lexicon, coverage-proved	—

7. Provenance and Dates

- **Author and sole committer:** Chris Sharp chrissharp80@gmail.com.
- **Private development repository** `chrissharp80/emuqu-dev`: first commit 2026-08-16 (squashed from earlier work).
- **Public repository** `github.com/chrissharp80/emuqu` (PolyForm Strict 1.0.0): first commit 2026-09-08.

- **Earlier product name "Flow Recovery"** (the author's own records): app-generated `Flow_Recovery_<date>.pdf` session reports in the author's Google Drive dated 2026-02-15, 2026-02-17 and 2026-03-03; a "Flow Recovery - Bluetooth" screen recording dated 2026-03-23; and a "flow-recovery-audit.md" dated 2026-05-10.
- **First appearance in emuqu-dev history** of the files implementing these methods:

Component	First added
Fact catalogue (<code>FactCatalog</code>), value union (<code>FactValue</code>)	2026-08-16
Number verifier (<code>MetricsVerifier</code>)	2026-08-16
Output guard (<code>CoachVoiceGuard</code>)	2026-08-16
Input guard (<code>MedicalQueryGuard</code>)	2026-08-16
User-assertion detector (<code>UserAssertedValuesParser</code>)	2026-08-16
Copy linter (<code>Tools/copy_linter/lint.py</code>)	2026-08-16
Shared lexicon (<code>MedicalTermLexicon</code>)	2026-08-26
Coverage gate (<code>scripts/check_perimeter_sync.sh</code>)	2026-08-26

- **This paper:** v1.0, published 2026-10-08, describing source revision e028039 (2026-10-07).

References

1. Ji, Z., Lee, N., Frieske, R., et al. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys* 55(12).
2. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)* 33.
3. Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS* 36.
4. Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.
5. Gao, L., Dai, Z., Pasupat, P., et al. (2023). RARR: Researching and Revising What Language Models Say, Using Language Models. *Proceedings of ACL 2023*.
6. Min, S., Krishna, K., Lyu, X., et al. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of EMNLP 2023*.
7. U.S. Food and Drug Administration (2019). *General Wellness: Policy for Low Risk Devices*. Guidance for Industry and FDA Staff.
8. Apple Inc. *App Store Review Guidelines*, Guideline 1.4.1 (Physical Harm — medical apps). Public documentation.
9. Greshake, K., Abdelnabi, S., Mishra, S., et al. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*.

10. Inan, H., Upasani, K., Chi, J., et al. (2023). Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674.
11. Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C., Cohen, J. (2023). NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. *Proceedings of EMNLP 2023: System Demonstrations*.
12. Chapman, W. W., Bridewell, W., Hanbury, P., Cooper, G. F., Buchanan, B. G. (2001). A Simple Algorithm for Identifying Negated Findings and Diseases in Discharge Summaries. *Journal of Biomedical Informatics* 34(5).
13. Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3(4).
14. OpenAI (2023). Function calling. OpenAI API documentation.
15. Anthropic (2024). Tool use and prompt caching. Anthropic API documentation.
16. Wallace, E., Xiao, K., Leike, R., et al. (2024). The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208.

Appendix A: Parameter Table

Name	Value	Unit	Role
Registered namespace resolvers	26	count	Registry address space
Declared catalogue entries	238 (149 fixed sync, 8 fixed async, 56 parameterised, 6 composite, 19 action)	count	Catalogue size at e028039
Absence reasons	9	count	Closed taxonomy
<code>rateLimitMissRetries</code>	2	misses	Third identical miss per turn → <code>rateLimited</code>
<code>outputSizeCapBytes</code>	80,000	bytes	Per-result cap → <code>tooMuchData</code>
<code>defaultResolveBudgetSec</code>	2.0	s	Per-entry path: overrun discards value
<code>resolveWarnBudgetSec</code>	0.5	s	Slow-resolver warning
<code>watchdogWarnSec</code>	4.0	s	Action main-thread warning
Valid-range text granularity	month of lower bound	—	Parameter description
Compact read tools	21	count	Model-facing schema
Allow-listed action tools	18	count	Model-facing schema
Capability-index purpose cap	170	characters	Per tool line
BM25 k1, b	1.5, 0.75	—	Tool ranking
BM25 target K	40	tools	Retriever filters only above this
Grok schema cap	110	tools	Per-provider cap

Name	Value	Unit	Role
<code>maxToolCallsPerTurn</code>	8	calls	Over-budget round → all calls <code>rateLimited</code>
Stream publish interval	33	ms	Throttle; sentence-complete publishing
HR tolerance	5	bpm	Verifier
Power tolerance	15	W	Verifier
DFA $\alpha 1$ tolerance	0.10	—	Verifier
HR drift tolerance	1.5	percentage points	Verifier
TSB tolerance	2.0	load units	Verifier
ATL / CTL tolerance	3.0	load units	Verifier
ACWR tolerance	0.15	ratio	Verifier
RMSSD tolerance	8.0	ms	Verifier
Recovery tolerance	4.0 (0–100) / 1.0 (0–10)	points	Verifier, scale disambiguation
Sleep duration tolerance	5 (precise) / 30 (whole hours)	min	Verifier
Sleep efficiency tolerance	1.0	percentage points	Verifier
Spelled-number range	0–99, one decimal digit	—	Normaliser
Pending corrections cap	4	entries	Next-turn reminder
Correction-detector window	last 6 turns (user, non-local)	turns	Assertion detection
Numeric arg: <code>starting_atl</code> , <code>starting_ctl</code> , <code>daily_trimp</code>	0–10,000	TRIMP	Range check
Numeric arg: <code>gap_trimp</code>	(0, 10,000]	TRIMP	Range check
Numeric arg: <code>horizon_days</code>	1–365, integer	days	Range check
Numeric arg: <code>lat</code> / <code>lon</code>	–90–90 / –180–180	degrees	Range check
Numeric arg: <code>radius_m</code>	5–5,000	m	Range check
Numeric arg: <code>ordinal</code>	0–100,000, integer	index	Range check
Numeric arg: <code>seconds</code>	1–86,400, integer	s	Range check
Numeric arg: <code>max_results</code>	1–10, integer	results	Range check
Lexicon concepts	29 (+2 offensive)	count	Shared vocabulary
Languages	17	count	en + 16 shipped locales
<code>refuseBeforeSending</code> / <code>scrubFromOutput</code> / <code>neverInStaticCopy</code>	12 / 21 / 12	concepts	Group sizes

Name	Value	Unit	Role
Sentence terminators	. ! ? \n 。 ! ? , ? ...	—	Segmentation
Emphasis markers stripped on retry	`* _ ``	—	Output guard
Offensive-language recogniser confidence	0.6	probability	English-only entries
Offensive-language min words	3	words	English-only entries
Voice echo overlap	0.40	token fraction	Refusal echo trap
Linters prohibition window	3 lines (match line and 2 above)	lines	Exemption
Linters snippet fragment	first 40 chars, ≥ 3	characters	Exemption
Build-time English patterns	62	count	<code>user_facing_prohibited</code>
Coverage-gate minimum alternatives	100 (985 at e028039)	count	Vacuity guard
Consent schema version	8	—	Per-provider consent
Request audit size	10	entries	In-memory FIFO

Appendix B: Pseudocode

B.1 Registry tool resolution

```

resolveTool(name, argsJSON):
  entry = findEntry(toolName == name) or return missing(invalidParameter, "unknown tool")
  if entry is action:
    args = parse(argsJSON) keeping only declared params, numbers/bools -> strings
    for p in entry.params where p.required and p not in args:
      return gate(name, argsJSON, missing(invalidParameter, "missing required arg
p"))
    v = await entry.execute(args)
    return gate(name, argsJSON, v)
  key = entry.key, or entry.pattern with "$param" replaced by args[param]
    (missing/bad arg -> missing(invalidParameter))
  t0 = now; v = await resolveAsync(key); dt = now - t0
  if dt > 2.0: v = missing(internalError, "timed out")
  return gate(name, argsJSON, v)

gate(name, argsJSON, v):
  if v is missing:
    sig = name + "::" + argsJSON; count[sig] += 1
    return count[sig] > 2 ? missing(rateLimited, ...) : v
  if utf8len(envelope(v)) > 80000: return missing(tooMuchData, ...)
  return v

```

B.2 Composite (async path)

```

compositeAsync(entry, key):
  param = entry.key has "(" ? capture(entry.key, key) : nil
  deps = entry.dependencies with "$name" -> param
  values = { d: await resolveAsync(d) for d in unique(deps) }
  saved = prefetch; prefetch = prefetch u values
  r = entry.body(param, registry)      # body's registry.resolve(k) reads prefetch
first
  prefetch = saved
  return r                             # {present:[{key,value}], missing:
[{{key,reason,detail}}, status?}

```

B.3 App-state number verification

```

verifyAppState(text):
  (norm, rewrites) = normaliseSpelledNumbers(text)
  found = []
  for (pattern, actual, tol, fmt, negateOnSpelled) in metricChecks:
    for m in pattern.matches(norm):
      s = sentenceAround(m, norm)
      if s matches timeMarkers or targetMarkers: continue
      c = number(m.value); if negateOnSpelled and m.claim contains "negative": c = -
|c|
      if |c - actual| > tol: found += (metric, fmt(c), fmt(actual), m.claimRange)
  apply recovery-scale rule and sleep sentence filters
  mapped = [remap(d.range, norm -> text, rewrites) for d in found]
  corrected = replace mapped ranges back-to-front in text with d.actual
  recordCorrections(mapped)           # dedupe (metric, claimed, actual); keep last 4
  return corrected

nextPromptVariableSections():
  block = takeAndClear(pendingCorrections)
  if block nonempty: append "# Last turn correction – DO NOT FABRICATE" + one line per
entry

```

B.4 User-assertion resolution

```

signals = {}
for msg in reversed(lastSixTurns.filter(user and not localOnly)):
  t = statementsOnly(normaliseSpelledNumbers(msg))
  for metric in knownMetricsLongestFirst:
    v = lastMatch(metric, connector required if metric in {sleep, recovery}, no clock
times)
    if v and metric not in signals.values: signals.values[metric] = v; blank matches
fold aliases into canonical if canonical absent
  signals.contradicted |= containsPhrase(t) or startsWithCorrectionReply(t)
  signals.override     |= containsOverridePhrase(t)
if signals nonempty: prompt += userStatedBlock(signals)
if not (signals.contradicted or signals.override): prompt += dashboardLoadBlock

```

B.5 Input guard

```

onSend(text):
  t = trim(text); if empty: reject
  trig = first of [selfHarm, emergency, injury, rhythm, dangerJudgement] whose regexes
  match t
  if trig and (flagOn or trig == selfHarm):
    store(userTurn(t, localOnly=true), assistantTurn(replyFor(trig, appLanguage),
    localOnly=true))
    return # no provider, no consent prompt
  continue to consent check and provider send
history(turns) = turns.filter(not localOnly and not emptyAssistant)

```

B.6 Streaming output guard

```

onDelta(chunk):
  buf += chunk
  if now - lastPublish < 33ms: return
  (complete, tail) = splitAtLastTerminator(buf) # decimal-aware
  if complete empty: return
  buf = tail
  visible += scrub(complete)
  if roundHasNoToolCall and published text has terminator: speakableCursor = len(visible)
onRoundEnd / onError / onStop:
  visible += scrub(buf); buf = ""
speak(utterance):
  synthesise(preamble + verify(scrub(utterance)))

```

B.7 Coverage gate

```

alts = [compile(latin ? "\b(?:\"+a+)\b" : a) for a in literals(latin/unbounded arrays of
lexicon*)]
fail(2) if any alt matches "" or len(alts) < 100
for p in buildTimePatterns:
  w = witness(p) # see 3.8 step 3
  if w == "": continue
  fail(2) if not p matches w
  gaps += p if no alt matches w
fail(1) if gaps else pass

```

Appendix C: Source Map

Method	Files (repository-relative)
Entry kinds, one-line-one-fact, namespace walk, pattern matching	Emuqu/Sources/Assistant/Facts/FactCatalog.swift
Key grammar	Emuqu/Sources/Assistant/Facts/FactKey.swift
Value union, absence taxonomy, availability, envelope	Emuqu/Sources/Assistant/Facts/FactValue.swift

Method	Files (repository-relative)
Registry, rails, schema, hash, composites, timeout race	Emuqu/Sources/Assistant/Facts/FactResolverRegistry.swift
Numeric argument ranges	Emuqu/Sources/Assistant/Facts/AppFactResolver+NumericArguments.swift
Namespace registration; entries	Emuqu/Sources/Assistant/Facts/AppFactResolver.swift, AppFactResolver+*.swift
Provenance flags (data_source, comparison_safe)	AppFactResolver+TrainingProjections.swift, AppFactResolver+WorkoutLiveDerived.swift, AppFactResolver+Sleep.swift
Compact tools, allow-list, capability index	Emuqu/Sources/Assistant/Facts/CompactToolRouter.swift, CompactToolRouter+Resolvers.swift
BM25 retrieval	Emuqu/Sources/Assistant/Facts/ToolRetriever.swift
Tool loop, call budget, chat verification, fallback	Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Tools.swift
On-device dispatcher	Emuqu/Sources/Assistant/Providers/AppleToolDispatcher.swift
Number verifier, correction buffer	Emuqu/Sources/Assistant/MetricsVerifier.swift
Voice verification and scrub order	Emuqu/Sources/Assistant/VoiceConversationController+Speech.swift
User-assertion detector	Emuqu/Sources/Assistant/UserAssertedValuesParser.swift
Prompt composition (cache suppression, corrections)	Emuqu/Sources/Assistant/Providers/AIProvider+SystemPrompt.swift; rules A–D in AIProvider+SystemPromptText.swift
Lexicon and groups	Emuqu/Sources/Assistant/MedicalTermLexicon.swift, MedicalTermLexicon+Injury.swift, MedicalTermLexicon+MetricVerdict.swift
Input guard	Emuqu/Sources/Assistant/MedicalQueryGuard.swift; call site Emuqu/Sources/Assistant/ViewModel/AssistantViewModel.swift; echo trap VoiceConversationController+Audio.swift
localOnly turns	Emuqu/Sources/Assistant/Providers/AIProvider.swift (ChatTurn); Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Context.swift
Output guard, offensive list	Emuqu/Sources/Assistant/CoachVoiceGuard.swift
Streaming buffer	Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Context.swift (StreamTextBuffer)
Final pass	Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Routing.swift
Copy linter and configuration	Tools/copy_linter/lint.py, Tools/copy_linter/prohibited_terms.json

Method	Files (repository-relative)
Coverage gate	scripts/check_perimeter_sync.sh
Consent	Emuqu/Sources/Assistant/ProviderConsentTracker.swift
Tests cited	EmuquTests/MedicalQueryGuardTests.swift, MedicalQueryGuardTests+Injury.swift, CoachVoiceGuardTests*.swift, FactCatalogValidationTests.swift
Validation register	Tools/science_register/register.json